Indian Institute of Technology, Bhilai

CSL351: Computer Networks

Assignment - 8 (Implement a New Congestion Control Algorithm in NS3)

Full Marks: 25 *Deadline: 23/03/2026, 11:59 PM*

Submission Instructions:

1) Answer all the questions.
2) Deliverables in a .zip:
    a) Submission Guidelines: For each part, create a separate folder. Upload all the folders and the Assignment Report in .zip.
    b) Readable Report [**3 Marks for report quality**] enumerating steps followed with screenshots for each of the important steps.
        i) Put the screenshots in the report for better clarity
        ii) Your report should include screenshots of your final results, and you are expected to explain in the report which function was used, which line of code set the particular socket to use a specific CCA, and how the congestion window was traced. Everything needs to be carefully explained in your report.
        iii) Explain everything that has been asked in your report and highlight the important parts, and attach each and every code file, script or anything that you used to complete the tasks.
    c) For all the plots, write the inferences that you have observed.
3) Follow the instructions given in the question carefully.
4) You can use LaTeX/Word to make the PDF.
5) The naming of the zip file should strictly follow the given format: <Roll_No>_<Name>_<Assignment No>. If your name is Alex, your roll number is B23CS055, then the filename should be: B23CS055_Alex_06.zip
6) **Any plagiarism case will be considered an unethical practice, and appropriate action will be taken against them.**

Objectives:

1. **Comprehend and Differentiate TCP Variants:**
    a. Explore and understand the workings of different TCP congestion control algorithms, including their characteristics, strengths, and weaknesses.

   b. Compare algorithms from loss-based, delay-based, and hybrid categories, with an emphasis on TCP Cubic and its performance in various scenarios.
2. **Analyze TCP Congestion Dynamics:**
   a. Simulate different TCP variants in NS-3 to study their behavior in response to network congestion.
   b. Understand how congestion window dynamics, such as slow start, congestion avoidance, and cwnd reductions, affect throughput and latency.
3. **Evaluate the Impact of Network Parameters:**
   a. Investigate the effects of varying bandwidth, latency, and MTU size on TCP performance metrics, including throughput, RTT, and congestion window.
   b. Analyze how these parameters influence different congestion control algorithms under similar network conditions.
4. **Implement and Test a New Congestion Control Algorithm:**
   a. Develop a new TCP Congestion Control Algorithm (CCA) by enhancing the slow start phase of TCP NewReno with Hystart from TCP Cubic.
   b. Evaluate the performance of the new algorithm against TCP NewReno using metrics like throughput, RTT, congestion window, and ssthresh.
5. **Assess Fairness and Scalability:**
   a. Measure the fairness of TCP variants using Jain's Fairness Index under varying flow counts (4, 8, 16, 20).
   b. Compare fairness between scenarios with all flows using TCP NewReno and scenarios with mixed flows (NewReno and the new algorithm).
6. **Visualize and Interpret Results:**
   a. Generate and interpret graphs of throughput, congestion window, RTT, and fairness index using tools such as Matplotlib or Gnuplot.
   b. Create detailed reports with step-by-step explanations, including screenshots of results and code, highlighting key observations and inferences.

Making a New TCP Congestion Control Algorithm in NS3 **[22 Points]**

In this question, you need to implement a new TCP Congestion Control Algorithm (CCA) and evaluate its performance and fairness (you can use Demo_2.cc and demo.cc for your reference).

1. You need to create a new CCA named Tcp<Your_first_name>, and you need to try to improve TcpNewReno's performance. **[1 Point]**
2. You need to focus only on the SlowStart phase, and to improve it, replace Reno's slow start with TCP Hystart (whose implementation can be found in src/internet/model/tcp-cubic.cc, ns-3's Tcp Cubic implementation). **[6 Points]**
3. You aim and try to make the slow start phase more intelligent with better exit points by doing this (more about Hystart at Reference a), but once you have done this, you need to evaluate your algorithm's performance, so consider the dumbbell topology in demo.cc, make all flows TCP and evaluate standard metrics like throughput, congestion window, ssthresh, and RTT variation between your algo and NewReno. **[4 Points]**
4. Also, calculate the value of Jain's fairness index (more about this at Reference b) by varying the number of flows as 4,8,16,20. In the first run of your experiment, all flows will use TcpNewReno, which will serve as your

baseline for comparison. In the second run, half of the flows will use TcpNewReno, while the other half will use your new CCA. Create a table like the one shown below and populate it with the fairness values achieved in each case. **[5 Points]**

Number of flows	4	8	16	20
NewReno & Your_CCA	<fairness_index>	<fairness_index>	<fairness_index>	<fairness_index>
NewReno	<fairness_index>	<fairness_index>	<fairness_index>	<fairness_index>

5. Plot the necessary graphs for all the metrics (throughput, congestion window, ssthresh, rtt) collected. **[3 Points]**
6. Finally, justify and explain the results of the comparison obtained. **[3 Points]**

References:

1. Sangtae Ha and Injong Rhee. 2011. Taming the elephants: New TCP slow start. Comput. Netw. 55, 9 (June, 2011), 2092–2110. https://doi.org/10.1016/j.comnet.2011.01.014
2. Jain's Fairness Index (https://doi.org/10.48550/arXiv.cs/9809099)

** **ns-3 version should be >=3.42**

Check Web sources for more information.